ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Т.А. Суетина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Основы создания 2D персонажа в Unity

по курсу: Мультимедийный практикум

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4329		Д.С. Шаповалова
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Цель работы:

Изучить базовые принципы создания управляемого 2D-персонажа в игровом движке Unity 3D. Освоить настройку физической модели (Rigidbody2D, Collider2D), работу с анимациями (Sprite Sheet, Animator, Blend Tree) и написание скриптов управления движением и прыжками.

2. Задание:

Создать анимированного управляемого 2D персонажа. Он может стоять, бегать, прыгать. Все это сопровождается соответствующими анимациями.

3. Ход работы:

3.1. Подготовка проекта и сцены

Создан новый проект в Unity с настройками по умолчанию для 2D.

В папке Assets создана подпапка Sprites для хранения графических ресурсов. Импортированы спрайты: Platform.png, Idle.png, Run.png, Jump.png.

Создан объект платформы: спрайт Platform перетащен на сцену, добавлен компонент Box Collider 2D для физической обработки столкновений.

Создан пустой объект Character. Добавлены компоненты:

Rigidbody 2D: включена опция Freeze Rotation Z (для запрета вращения) и Interpolate (для плавности отрисовки).

Sprite Renderer: для отображения графики персонажа.

Box Collider 2D: настроен по размеру спрайта, чтобы персонаж не проваливался сквозь платформы.

Animator: для управления состояниями анимации (параметр Animate Physics включен для синхронизации с физикой).

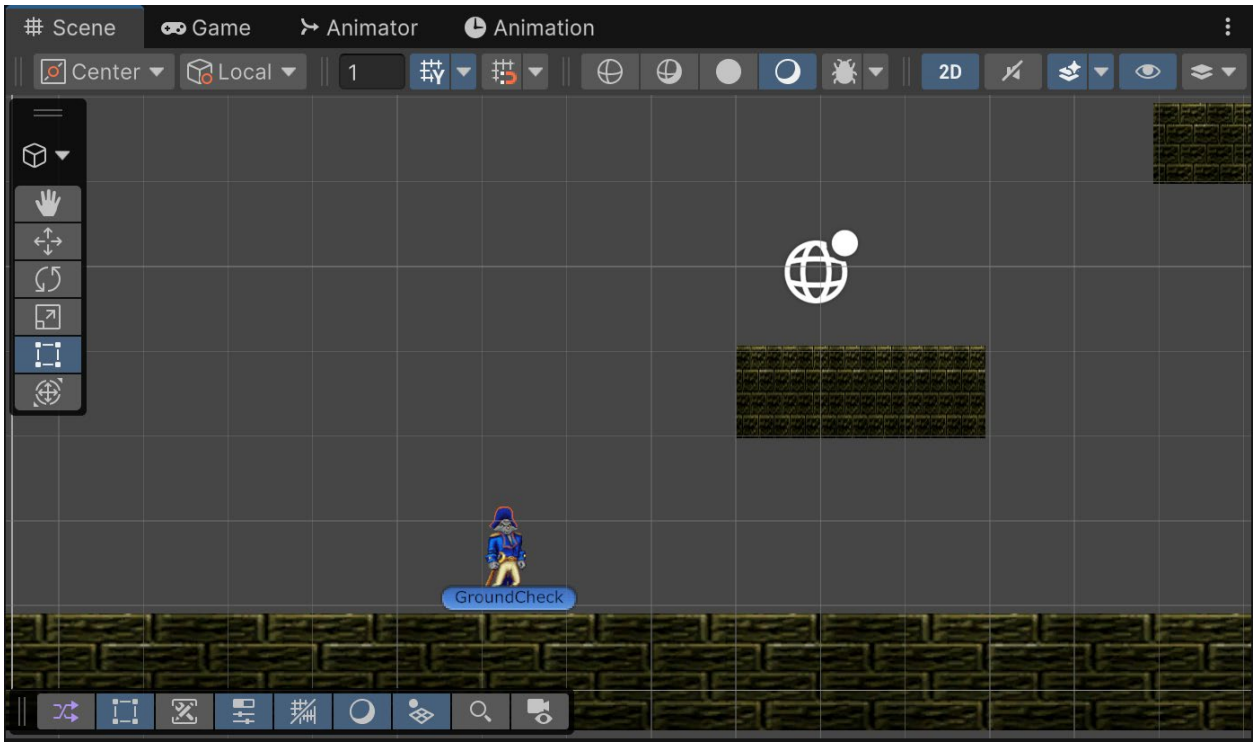

Рисунок 1 – Создание сцены

3.2. Настройка спрайтов и анимаций

Для всех спрайтшитов (Idle, Run, Jump) в Inspector изменён параметр Sprite Mode на Multiple.

Через окно Sprite Editor выполнена нарезка спрайтов на отдельные кадры (режим Slice → Automatic/Grid).

Созданы анимационные клипы в окне Animation:

Idle: перетащены кадры Idle_0 – Idle_7, скорость анимации установлена на 0.12.

Run: перетащены кадры Run_0 – Run_9, скорость анимации установлена на 0.15.

Jump: создано 7 отдельных клипов (Jump1 – Jump7), каждый содержит один соответствующий кадр из спрайтшита прыжка.

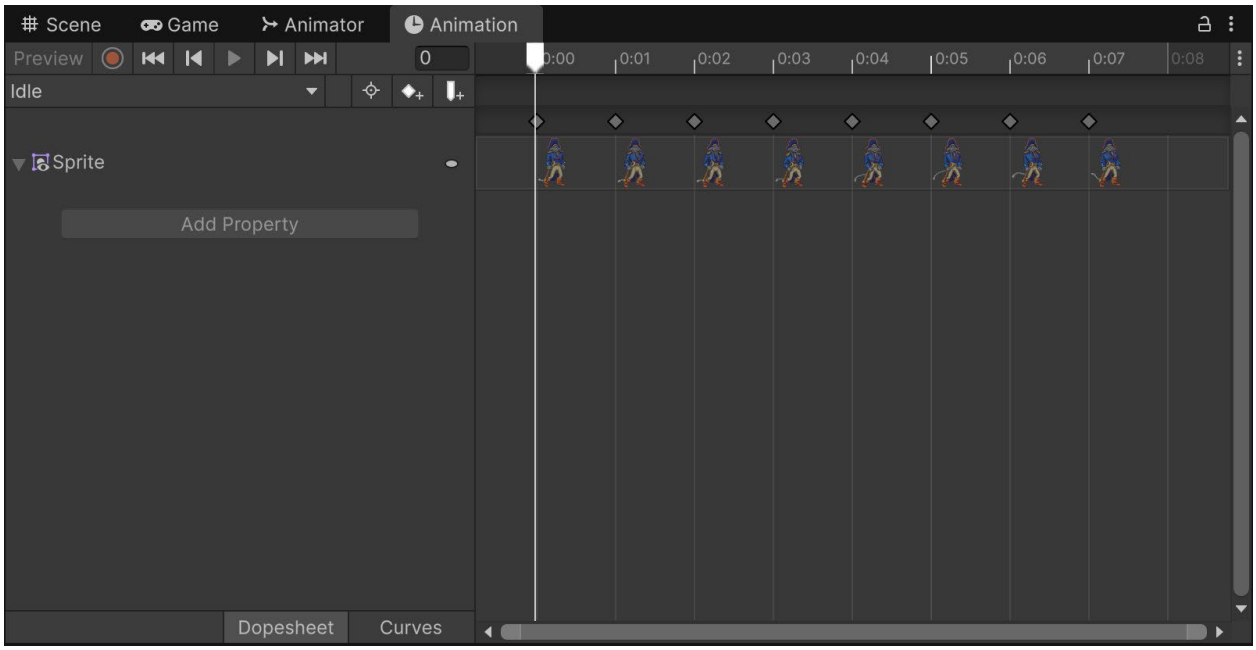

Рисунок 2 – Настройка спрайтов анимации

3.3. Настройка контроллера анимации (Animator)

Создан файл CharacterController (Animator Controller) и назначен компоненту Animator персонажа.

Добавлены параметры управления:

Speed (Float) — для переключения между покоем и бегом.

Ground (Bool) — для определения нахождения на земле.

vSpeed (Float) — вертикальная скорость для анимации прыжка.

Настроены переходы (Transitions):

Idle ↔ Run: условие по параметру Speed (больше/меньше 0.01), флаг Has Exit Time отключен для мгновенной реакции.

Any State → Jump: условие Ground = false.

Jump → Idle: условие Ground = true.

Для анимации прыжка создан Blend Tree:

Параметр управления изменён на vSpeed.

Добавлены 7 полей Motion для анимаций Jump1–Jump7.

Вручную настроены пороги (Thresholds) вертикальной скорости для каждого кадра, чтобы анимация соответствовала фазе взлёта или падения.

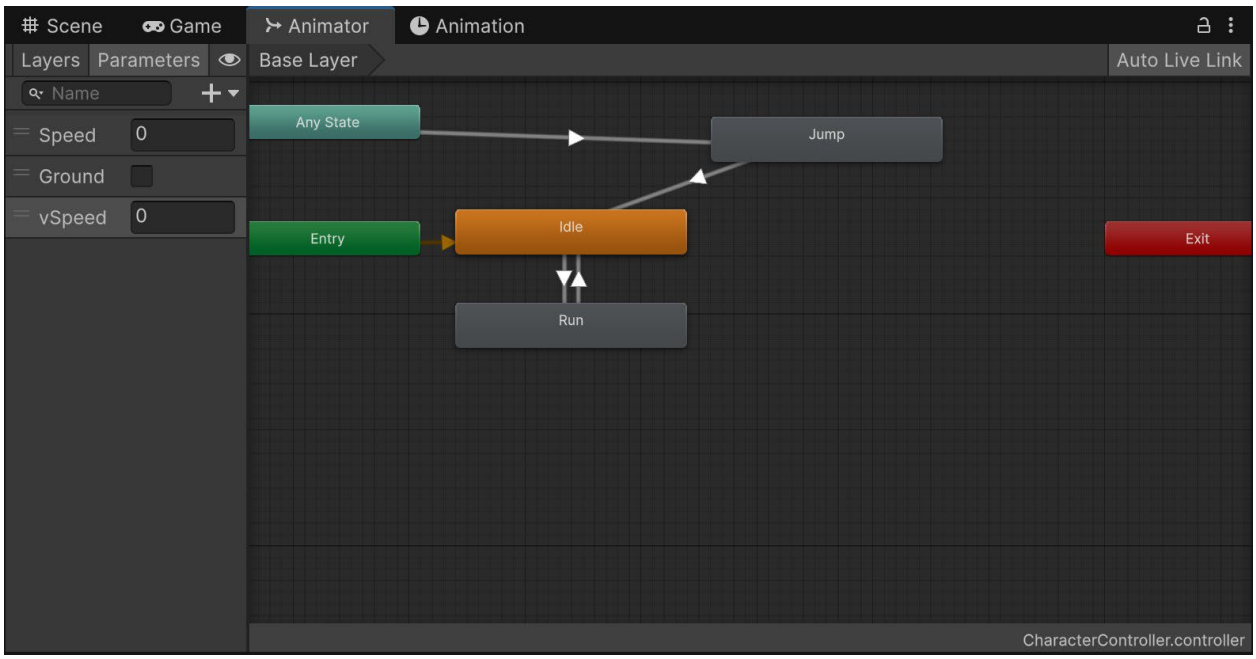

Рисунок 3 – Настройка переходов между анимациями

3.4. Программная реализация управления

Создан скрипт CharacterControllerScript.cs.

Реализована логика движения в методе FixedUpdate:

Считывание ввода по горизонтальной оси.

Изменение скорости Rigidbody2D через velocity.

Обновление параметра Speed в Animator.

Функция Flip(): разворот персонажа через инверсию localScale.x в зависимости от направления движения.

Реализована логика прыжка в методе Update:

Проверка условия isGrounded и нажатия клавиши действия (Space).

Применение силы вверх через AddForce.

Реализована проверка земли:

Создан дочерний объект GroundCheck в области ног персонажа.

В скрипте используется Physics2D.OverlapCircle для проверки пересечения объекта GroundCheck со слоем Ground.

3.5. Настройка физики и слоёв

Создан новый слой Ground (User Layer 8).

Объектам платформ назначен слой Ground.

В скрипт персонажа передана ссылка на слой whatIsGround.

В настройках проекта (Edit → Project Settings → Physics 2D) изменена гравитация Gravity Y на -30 для обеспечения реалистичной высоты и скорости прыжка.

3.6. Тестирование

Запущена сцена в режиме Play.

Проверено перемещение персонажа влево/вправо (клавиши A/D или стрелки).

Проверена корректность переключения анимаций Idle/Run.

Выполнен прыжок (клавиша Space): подтверждено изменение анимации через Blend Tree в зависимости от вертикальной скорости, персонаж не вращается и корректно приземляется на платформы.

ВЫВОД

В ходе работы создан полностью функциональный 2D-персонаж с физической моделью и системой анимаций, реагирующий на ввод пользователя. Освоены инструменты Unity для работы с 2D-физикой, анимационными состояниями и скриптом поведения.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ CharacterController:

```
using UnityEngine;
using UnityEngine.InputSystem;

public class CharacterControllerScript : MonoBehaviour
{
    public float maxSpeed = 10f;
    public float jumpForce = 7f;  // ← Сила прыжка

    private bool isFacingRight = true;
    private Animator anim;
    private Rigidbody2D rb;

    private bool isGrounded = false;
    public Transform groundCheck;
    private float groundRadius = 0.2f;
    public LayerMask whatIsGround;

    [SerializeField] private InputActionReference moveAction;
    [SerializeField] private InputActionReference jumpAction; // ← Добавили

    private bool jumpRequested = false; // ← Флаг для обработки прыжка

    private void Awake()
    {
        anim = GetComponent<Animator>();
        rb = GetComponent<Rigidbody2D>();

        if (groundCheck == null)
            Debug.LogError("❌ groundCheck не назначен!");
    }

    private void OnEnable()
    {
        if (moveAction?.action != null) moveAction.action.Enable();
        if (jumpAction?.action != null)
        {
            jumpAction.action.Enable();
            jumpAction.action.performed += OnJump; // ← Подписка на нажатие
        }
    }

    private void OnDisable()
    {
        if (moveAction?.action != null) moveAction.action.Disable();
        if (jumpAction?.action != null)
        {
            jumpAction.action.Disable();
            jumpAction.action.performed -= OnJump;
        }
    }

    // ← Метод вызывается при нажатии кнопки прыжка
    private void OnJump(InputAction.CallbackContext context)
    {
        if (isGrounded)
            jumpRequested = true;
    }

    private void FixedUpdate()
    {
        if (groundCheck == null) return;
```

```
        // Проверка земли
        isGrounded = Physics2D.OverlapCircle(groundCheck.position, groundRadius,
whatIsGround);
        anim.SetBool("Ground", isGrounded);
        anim.SetFloat("vSpeed", rb.linearVelocity.y);

        // Обработка прыжка (в FixedUpdate для физики)
        if (jumpRequested && isGrounded)
        {
            Jump();
            jumpRequested = false;
        }

        // Движение (можно и в воздухе)
        Vector2 moveVector = moveAction?.action?.ReadValue<Vector2>() ??
Vector2.zero;
        float move = moveVector.x;

        anim.SetFloat("Speed", Mathf.Abs(move));
        rb.linearVelocity = new Vector2(move * maxSpeed, rb.linearVelocity.y);

        if (move > 0 && !isFacingRight)
            Flip();
        else if (move < 0 && isFacingRight)
            Flip();
    }

    // ← Метод прыжка
    private void Jump()
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpForce);
    }

    private void Flip()
    {
        isFacingRight = !isFacingRight;
        Vector3 theScale = transform.localScale;
        theScale.x *= -1;
        transform.localScale = theScale;
    }

    private void OnDrawGizmosSelected()
    {
        if (groundCheck != null)
        {
            Gizmos.color = Color.yellow;
            Gizmos.DrawWireSphere(groundCheck.position, groundRadius);
        }
    }
        }
```